Tetris 2.0: Testing

This project uses four types of tests: **unit**, **integration**, **regression**, and **acceptance**. There are **202 automated tests** in 7 files.

Test type	What it answers	Files	Tests
Unit	Does each rule and helper work on its own?	`test_tetris_auth.py`, `test_tetris_features.py`, `test_tetris_render.py`, `test_tetris_extras.py`, `test_tetris_cloud.py` (first classes)	159 for unit and integration together
Integration	Do the parts work together?	`test_tetris_cloud.py`, `test_tetris_features.py` (play loop), `test_tetris_extras.py` (B key)	included in the 159

1. Unit Tests

Purpose: test each rule or helper by itself, with no window and no network, so a failure points to one place.

File	Tests	What it checks
`test_tetris_auth.py`	56	Username and password rules, hashing, registering, signing in, loading damaged files, score file paths, Sign In and Sign Up screen behavior
`test_tetris_features.py`	43	Pause, stats, combos, back-to-back, level up, T-spins, lock delay, animations, themes, settings files, sound generation, layout
`test_tetris_render.py`	10	Score file load and save, drawing helpers, ghost piece

2. Integration Tests

Purpose: check that separate parts work correctly when connected.

File	What it checks
`test_tetris_cloud.py`	A real score server on a free port with the real client: API keys, bad requests, saving across a server restart, merging local and cloud scores, and game over sending the score to the server
`test_tetris_features.py` (`PlayLoopSmokeTest`)	Pausing, themes, muting, game over, and restart through the real play loop with fake key events
`test_tetris_extras.py`	The B key reaching the bomb, and music following the game state

Example: play until game over with cloud sync on, then check that the server holds the

3. Regression Tests

Purpose: make sure changes don't quietly break what already works. Run them after every change and before every push.

`test_tetris_regression.py` has three parts:

Part	What it does
Golden values	Locks the scoring tables, gravity curve, lock delay, bomb limits, account limits, and music tempo. Changing one on purpose means updating its test
Past trouble spots	Damaged files, usernames with path tricks, zero scores, double counting in sync, pause freezing everything, bombs on an empty floor, hold rules, unknown sounds, passwords never stored
Random	25 seeded random games of 1,500 moves each. After every move the piece must be in a valid spot, the score must never drop, the queue must

4. Acceptance Tests

Purpose: check that the game does what a player (or the instructor) asked for. Each test is a short Given / When / Then story using only things a player can see or do.
`test_tetris_acceptance.py` has 20 tests, named after requirements R1 to R10 in `SOFTWARE_DESIGN.md`.

Group	Stories
Accounts (R1)	A new player signs up; a returning player signs in; a wrong password and an unknown name give the same message; sign up explains what to fix; passwords are never written down
Gameplay (R2 to R6, R8)	A piece falls and can be steered; a hard drop lands and the next piece appears; the player sees 3 next pieces and can hold once per piece; a Tetris scores 800; 10 lines raise the level; a Tetris earns a bomb and the bomb clears rows; pause and resume; game over with stats

5. How to Run the Tests

From the game folder:

```
pip install pygame
python3 -m unittest test_tetris_render test_tetris_auth test_tetris_features test_tetris_cloud test_tetris_extras test_tetris_regression test_tetris_acceptance
```

Run one test type:

```
python3 -m unittest test_tetris_regression
python3 -m unittest test_tetris_acceptance
```

Run one test:

```
python3 -m unittest test_tetris_extras.BombTests.test_bomb_scores_and_announces
```

The tests use dummy video and audio drivers, so no window opens and no sound plays. They also use temporary folders, so they never touch your real `users.json`, `scores`, or `settings`.